

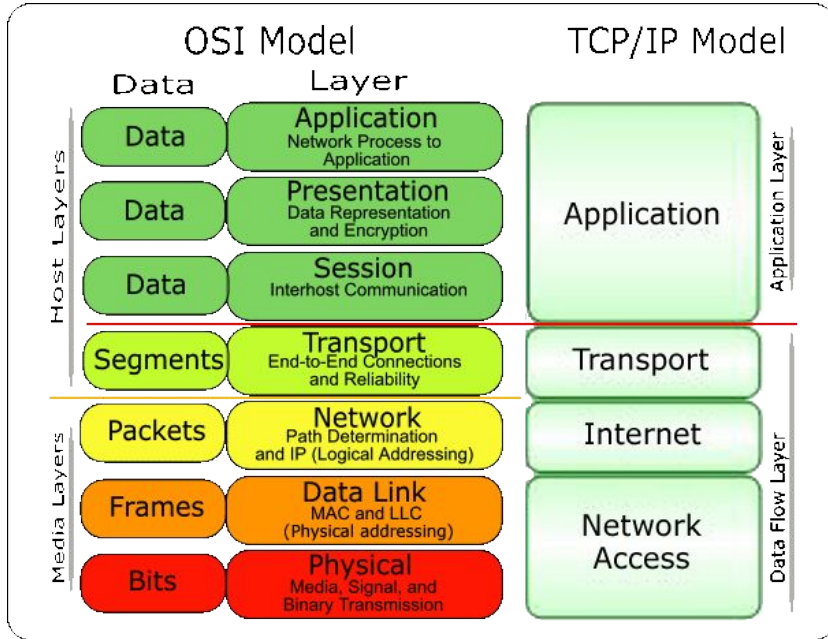
Introducción al protocolo HTTP

Programación en Entornos de Red

GRADO EN INGENIERÍA BIOMÉDICA 2017-2018

Juan Gonzalez Gomez <juan.gonzalez.gomez@urjc.es>,
Agustín Santos <agustin.santos@urjc.es>

Modelo de Referencia OSI y TCP/IP

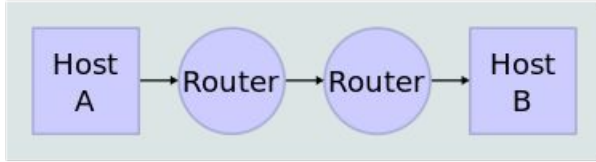


El modelo de referencia OSI es un modelo teórico de lo que podrían ser las diferentes capas que intervienen en las comunicaciones en redes de ordenadores.

Es similar a TCP/IP pero desglosa en más capas las que existen en TCP/IP, detallando con más precisión las diferentes capas.

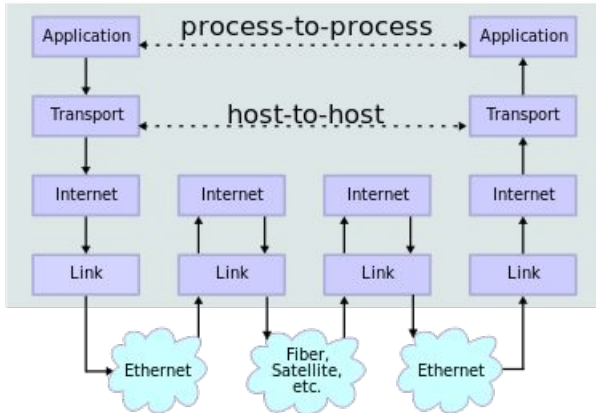
Flujo de datos y modelo de red en TCP/IP

Network Topology



Host A y Host B se encuentran ambos conectados a sendos routers que a su vez se haya conectados entre ellos por Internet.

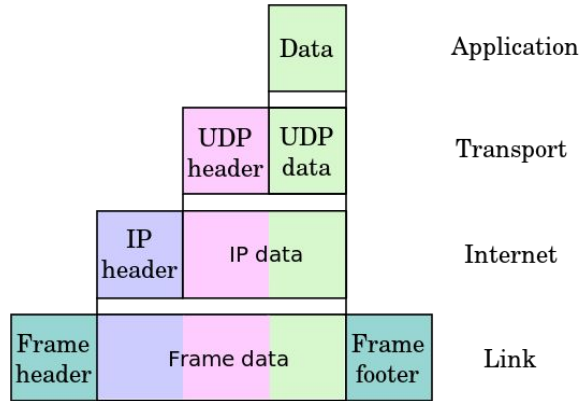
Data Flow



A nivel de aplicación, los datos se envían entre procesos. Pero en realidad, estos datos son recogidos en el cliente y se van entregando a las capas inferiores de los protocolos de comunicación para ser enviados al cliente.

Encapsulación en paquetes TCP/IP

Los datos se envían desde el nivel de Aplicación desde el cliente al servidor.



Los datos se trocean en paquetes numerados en el nivel de transporte. Estos paquetes se unirán de forma ordenada en el destino. A los paquetes se les añade el puerto destino.

A los paquetes de transporte se les añade la dirección IP de origen y destino en la capa de red.

En la capa de enlace a los paquetes se les añade la dirección MAC (identificador único de una interfaz de red en el área local) del siguiente salto al que tiene que ir el paquete.

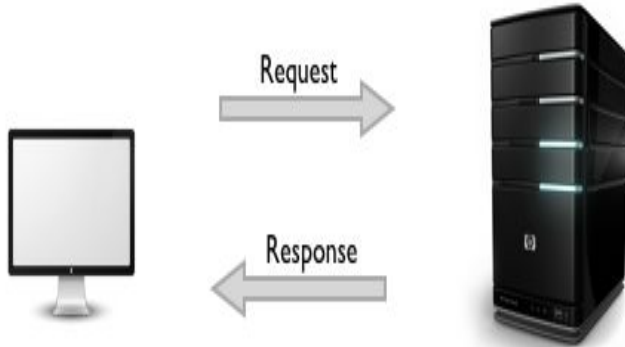
Protocolo HTTP

HTTP es un protocolo a nivel de aplicación de petición-respuesta basado en el modelo cliente servidor. Se utiliza en sistemas distribuidos y colaborativos de intercambio de información hipermedia pero es útil en otros escenarios. Es un protocolo sin estado (las peticiones son todas independientes).

Nació en 1990 y su última versión, HTTP/2, es de 2015. La versión principal es la HTTP/1.1 que se publicó de forma definitiva en 2014.

Las peticiones principales que soporta son: GET, HEAD, POST, PUT, DELETE.

Protocolo HTTP

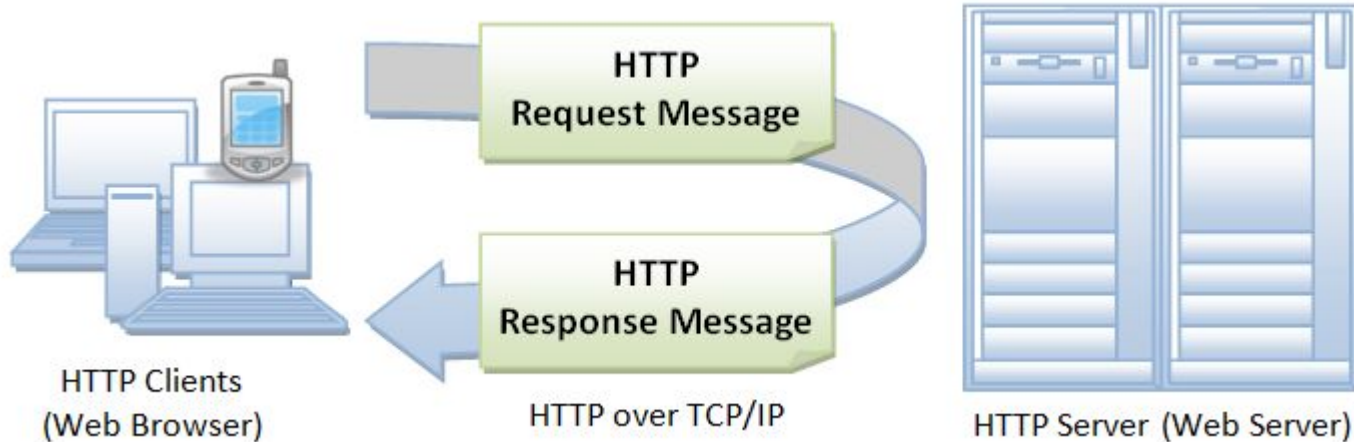


- Es un modelo cliente/servidor
- Utiliza el modelo de petición/respuesta
- Utiliza TCP, normalmente en el puerto 80
- Tiene dos partes: cabecera y contenido
- La primera línea de la petición es la orden o petición (el “verbo”)
- La primera línea de la respuesta es el código de estado.
- Hay varios tipo de peticiones (o de verbos)

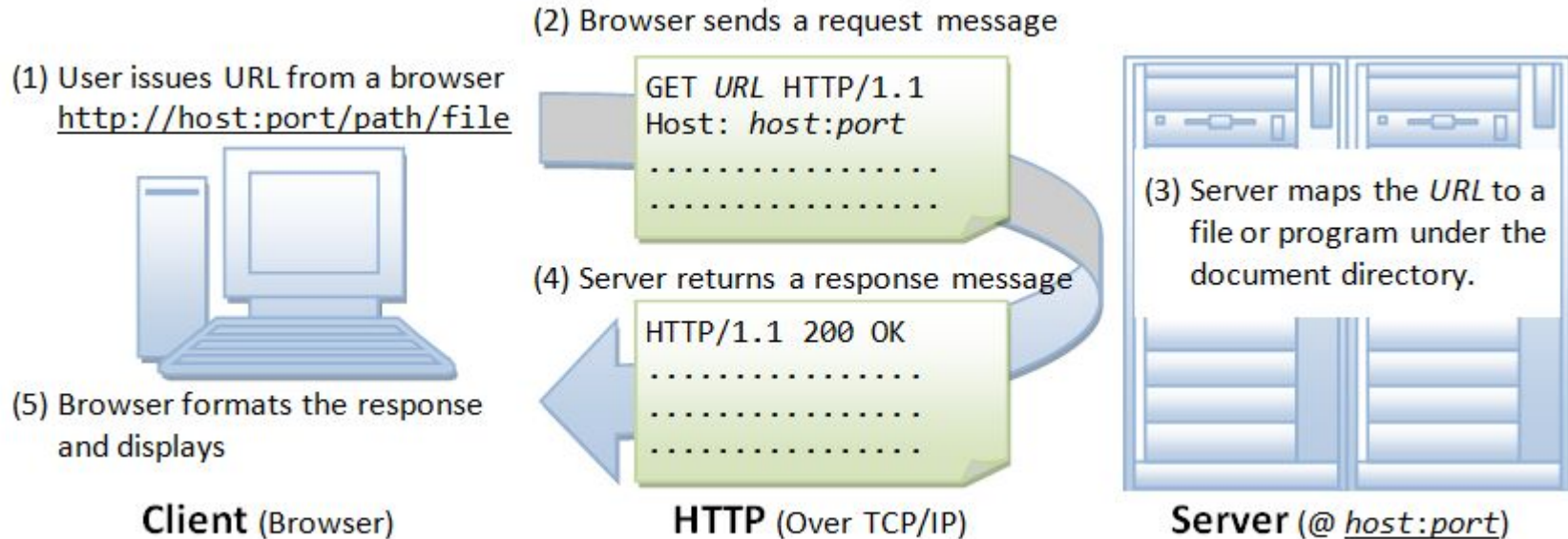
Propiedades HTTP

- **El cliente trabaja “sin conexión”.**
 - Pide algo y desconecta.
 - Si necesita algo más, vuelve a establece una conexión, pide otra cosa y desconecta
- **No depende de la naturaleza del contenido o “media”.**
 - No distingue entre ficheros de texto, html, imágenes, vídeos, etc.
 - Simplemente se ponen de acuerdo el cliente y el servidor
 - El tipo del contenido se especifica en el MIME-type
- **Trabaja “sin estado”**
 - El servidor no recuerda al cliente.

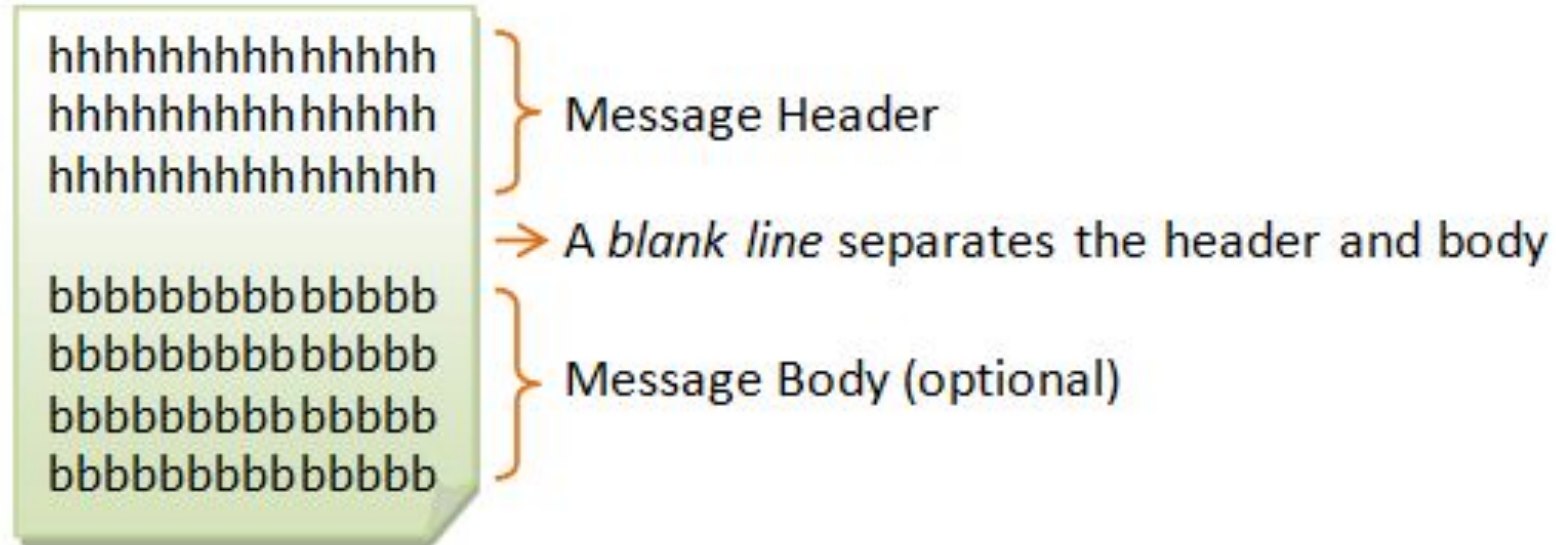
HTTP: Petición/Respuesta



HTTP: Contenido de los mensajes

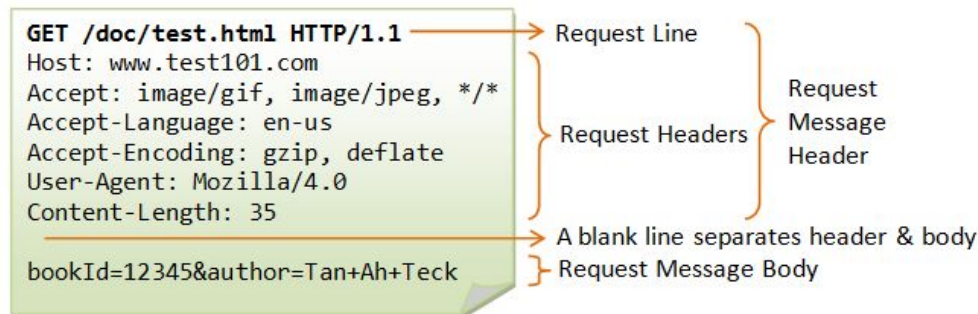
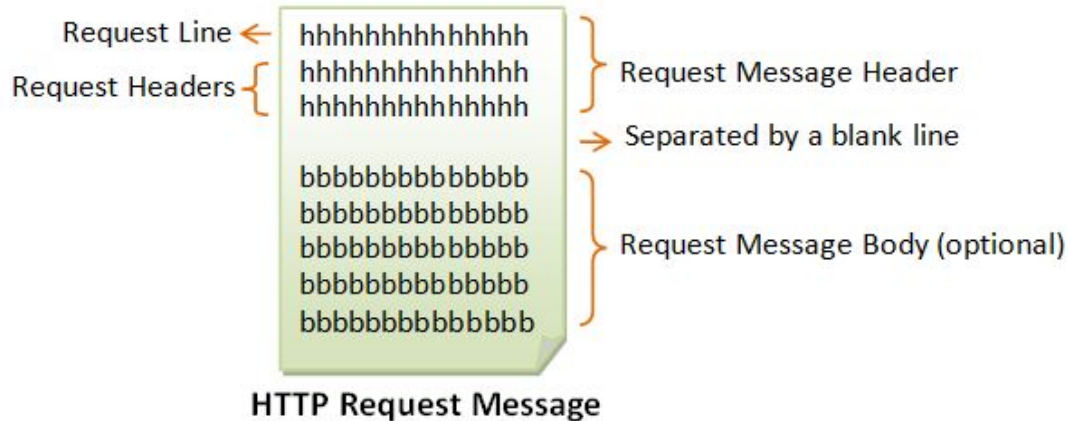


HTTP: Mensajes

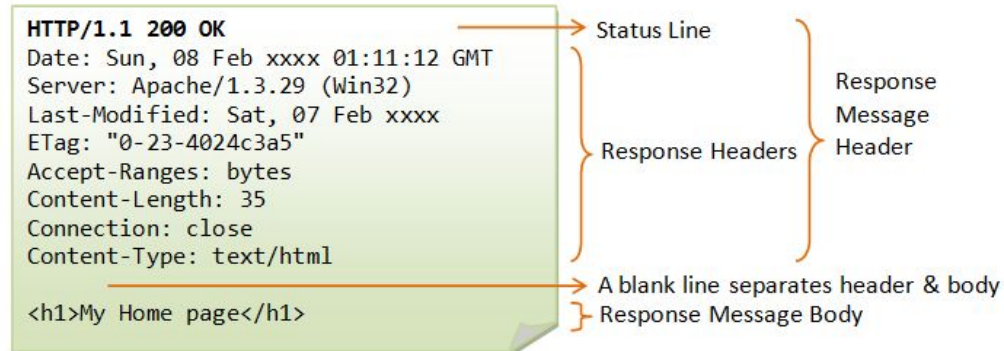
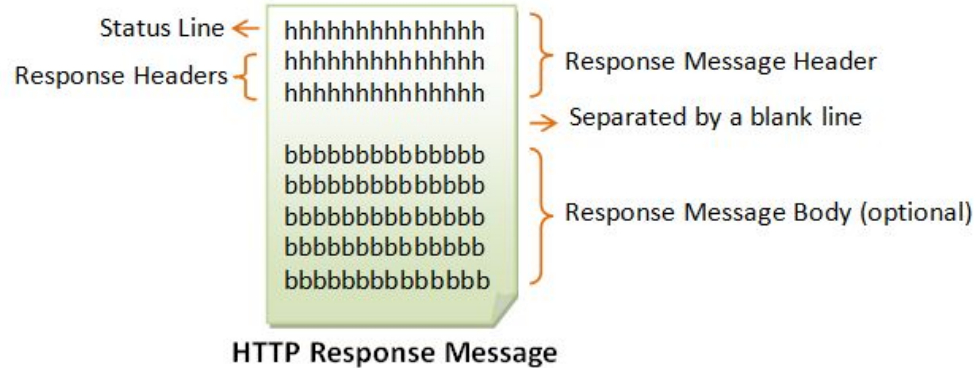


HTTP Messages

HTTP: Petición



HTTP: Respuesta



Probando, probando..

Para hacer nuestras pruebas, realiza lo siguiente:

1. Abre dos terminales.
2. En una de ellas, crea un directorio llamado “server” y entra dentro

```
mkdir server  
cd server
```

3. Arranca un servidor “de juguete” que tiene por defecto Python.

```
python3 -m http.server
```

Te dirá algo del estilo:

```
Serving HTTP on 0.0.0.0 port 8000 ...
```

Probando, probando..

Para hacer nuestras pruebas, realiza lo siguiente:

1. En el otro terminal prueba estos comandos (cambia la IP por la de tu máquina)

```
HEAD 212.128.254.50:8000
```

```
HEAD www.urjc.es
```

```
GET www.urjc.es
```

2. La orden HEAD te muestra la cabecera
3. La orden GET pide un recurso
4. Observa las trazas en tu servidor python

HEAD www.urjc.es

```
asantos@gamma:~$ HEAD www.urjc.es
200 OK
Cache-Control: public, max-age=900, stale-while-revalidate=1800, stale-if-error=3600
Connection: close
Date: Wed, 21 Feb 2018 17:17:43 GMT
Via: 1.1 varnish-v4
Age: 550
Server: Apache/2.4.10 (Debian)
Content-Type: text/html; charset=utf-8
Expires: Wed, 17 Aug 2005 00:00:00 GMT
Last-Modified: Wed, 21 Feb 2018 17:17:43 GMT
Client-Date: Wed, 21 Feb 2018 17:26:55 GMT
Client-Peer: 212.128.240.50:80
Client-Response-Num: 1
P3P: CP="NOI ADM DEV PSAi COM NAV OUR OTRo STP IND DEM"
Set-Cookie: BIGipServer~Servred~pool_www_80=188000448.20480.0000; path=/; Httponly
X-Cache: HIT
X-Cache-Hits: 239616
X-Content-Powered-By: K2 v2.8.0 (by JoomlaWorks)
X-Logged-In: False
X-Varnish: 30420061 30262785
```

Probando, probando..

1. En el segundo terminal prueba ahora el comando (cambia la IP por la de tu máquina)

```
GET 212.128.254.50:8000
```

2. La orden GET pide un recurso
3. Crea una página index.html y copia al directorio server
4. Vuelve a lanzar estos comandos GET

```
GET 212.128.254.50:8000
```

```
GET -f 212.128.254.50:8000/index.html
```

```
GET -U -f 212.128.254.50:8000/index.html
```

```
GET -U -S -f 212.128.254.50:8000/index.html
```

```
GET -E -f 212.128.254.50:8000/index.html
```


Qué información tenemos

```
asantos@gamma:~$ GET -E -f 212.128.254.50:8000/index.html  
GET http://212.128.254.50:8000/index.html  
User-Agent: lwp-request/6.15 libwww-perl/6.15
```

Nuestra petición

```
200 OK  
Date: Wed, 21 Feb 2018 18:14:07 GMT  
Server: SimpleHTTP/0.6 Python/3.5.2  
Content-Length: 98  
Content-Type: text/html  
Last-Modified: Wed, 21 Feb 2018 18:04:57 GMT  
Client-Date: Wed, 21 Feb 2018 18:14:07 GMT  
Client-Peer: 212.128.254.50:8000  
Client-Response-Num: 1
```

Cabecera de petición

Información de
respuesta

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h1>Una cabecera</h1>  
<p>Un texto cualquiera.</p>
```

El contenido

```
</body>  
</html>
```

Probando, probando..

1. Vuelve a lanzar estos comandos GET. Queremos producir un error

```
GET -E -f 212.128.254.50:8000/noexiste.html
```

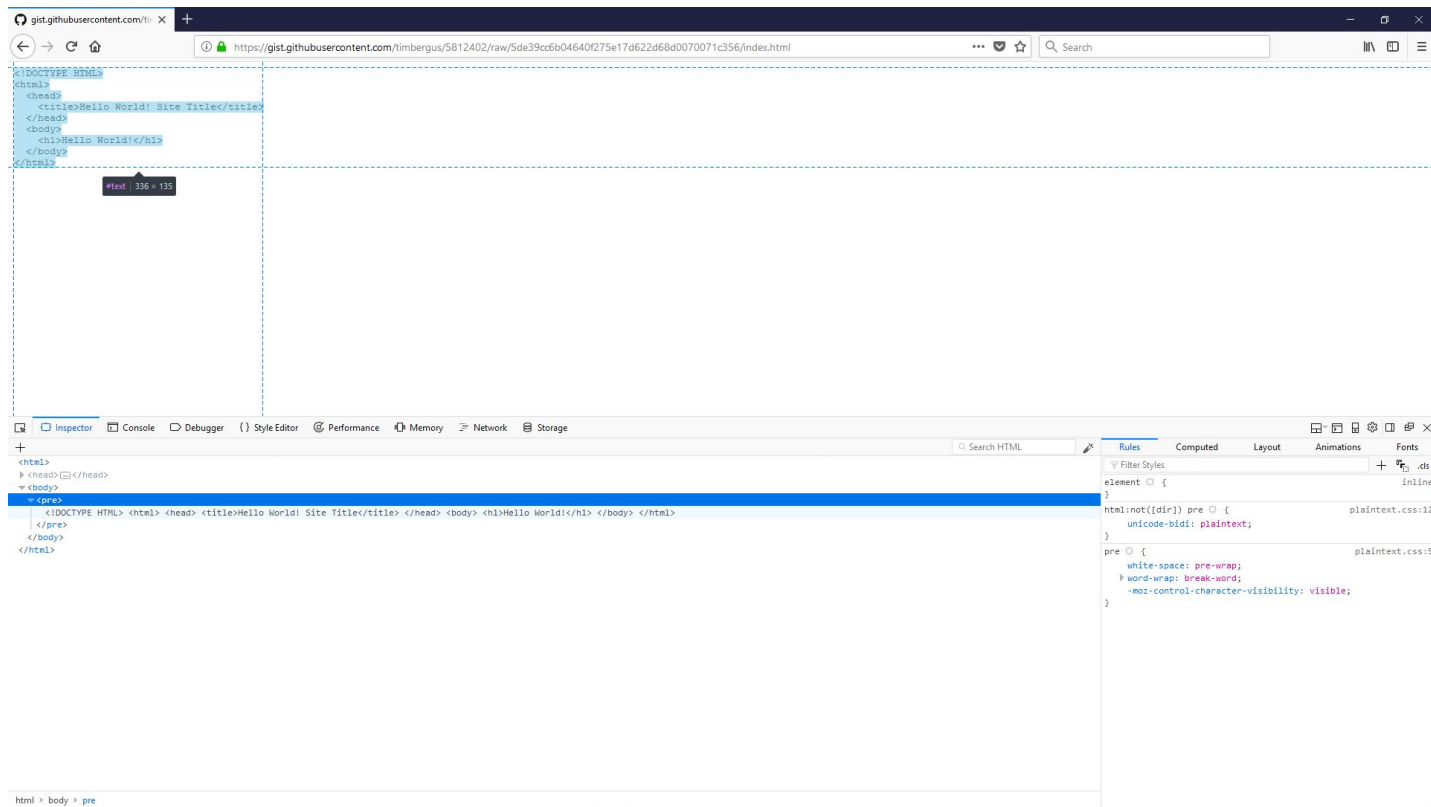
```
GET -E -f 212.128.254.50:8000/dir/index.html
```

2. Intenta encontrar el código de error.
3. Intenta acceder a las mismas páginas desde tu navegador
4. Observa los códigos de error que te genera el navegador

Web Developer tools

- Activa las Web Dev tools en tu navegador
 - Depende del navegador
 - Normalmente en el menú de herramientas
 - Se suele llamar Developer Tools
 - Prueba en varios navegadores

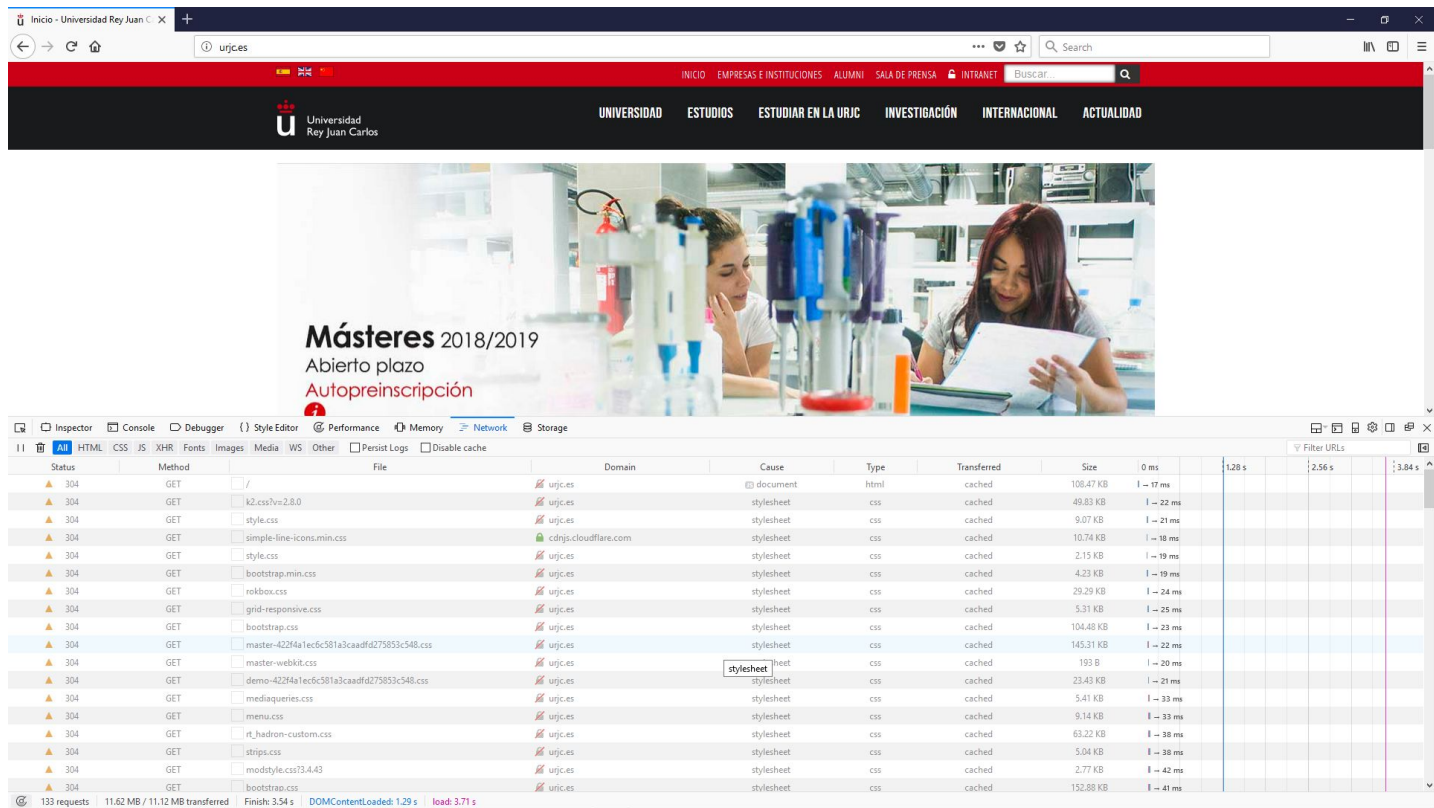
Analizando la página Web



Ejercicios

- Haz algunos de los ejemplos de:
 - https://www.w3schools.com/html/html_basic.asp
- Los ejemplos de la página pruébalos desde esa página.
- Después cópialos a un fichero html que esté en tu servidor python
- Prueba a abrirlos desde un navegador
- La idea es que pruebes a editar html, usando ficheros propios y servidos por el servidor http que tienes en tu máquina.
- Prueba los ejemplos con imágenes
 - https://www.w3schools.com/html/html_images.asp
- Mira las conexiones

Analizando las conexiones al servidor



The screenshot shows a web browser displaying the homepage of the University of Rey Juan Carlos (URJC). The page features a red header with navigation links and a main banner image of two students in a laboratory. Below the banner, the text "Másteres 2018/2019 Abierto plazo Autopreinscripción" is visible.

The Chrome DevTools Network tab is open, showing a list of HTTP requests. The table below represents the data shown in the Network tab:

Status	Method	File	Domain	Cause	Type	Transferred	Size	0 ms	1.28 s	2.56 s	3.84 s
304	GET	/	urjc.es	document	html	cached	106.47 KB	1 - 17 ms			
304	GET	k2.css?v=2.8.0	urjc.es	stylesheet	css	cached	49.83 KB	1 - 22 ms			
304	GET	style.css	urjc.es	stylesheet	css	cached	9.07 KB	1 - 21 ms			
304	GET	simple-line-icons.min.css	cdnjs.cloudflare.com	stylesheet	css	cached	10.74 KB	1 - 18 ms			
304	GET	style.css	urjc.es	stylesheet	css	cached	2.15 KB	1 - 19 ms			
304	GET	bootstrap.min.css	urjc.es	stylesheet	css	cached	4.23 KB	1 - 19 ms			
304	GET	rekbox.css	urjc.es	stylesheet	css	cached	29.29 KB	1 - 24 ms			
304	GET	grid-responsive.css	urjc.es	stylesheet	css	cached	5.31 KB	1 - 25 ms			
304	GET	bootstrap.css	urjc.es	stylesheet	css	cached	104.48 KB	1 - 23 ms			
304	GET	master-422f4a1ec6c581a3caadfd75853c548.css	urjc.es	stylesheet	css	cached	145.31 KB	1 - 22 ms			
304	GET	master-webkit.css	urjc.es	stylesheet	css	cached	193 B	1 - 20 ms			
304	GET	demo-422f4a1ec6c581a3caadfd75853c548.css	urjc.es	stylesheet	css	cached	23.43 KB	1 - 21 ms			
304	GET	mediaqueries.css	urjc.es	stylesheet	css	cached	5.41 KB	1 - 33 ms			
304	GET	menu.css	urjc.es	stylesheet	css	cached	9.14 KB	1 - 33 ms			
304	GET	rt_haaron-custom.css	urjc.es	stylesheet	css	cached	63.22 KB	1 - 38 ms			
304	GET	strips.css	urjc.es	stylesheet	css	cached	5.04 KB	1 - 38 ms			
304	GET	modstyle.css?3.4.43	urjc.es	stylesheet	css	cached	2.77 KB	1 - 42 ms			
304	GET	bootstrap.css	urjc.es	stylesheet	css	cached	152.88 KB	1 - 41 ms			

At the bottom of the Network tab, the summary shows: 133 requests, 11.62 MB / 11.12 MB transferred, Finish: 3.34 s, DOMContentLoaded: 1.29 s, load: 3.71 s.

Ejercicios para ir cerrando

- Prueba los ejemplos con enlaces (links)
 - https://www.w3schools.com/html/html_links.asp
- Haz dos páginas web e intenta que se llamen una a la otra
 - Sitúa ambas páginas dentro de tu mismo directorio y en tu servidor
- Trabajad en grupo. Haced dos o más páginas que se enlacen entre ellas.
 - Sitúa varias páginas en dos o más servidores diferentes.
 - Que tu página apunte a la página de un compañero
 - Que la página de tu compañero apunte a tu página
 - Haz una cadena de varias páginas enlazando tantas páginas como podáis de vuestros compañeros.
 - Haz una página web que contenga una imagen alojada en el servidor de un compañero.
 - Analiza las conexiones desde el Dev Tool de tu navegador

Hoy hemos aprendido

Funcionamiento de Http:

- Peticiones cliente-servidor,
- Utilización de verbos GET, HEAD
- Contenido habitual de las cabeceras
- Es un protocolo sin estado y sin conexión permanente

Funcionamiento de HTML:

- Lenguaje de marcado
- Tags (marcas) más habituales
- Sirve para describir la estructura de un documento.
- No es un lenguaje que “compute”